Design of high fidelity combinatorial libraries

November 5, 2026

Abstract

Recent tools for de novo design have revolutionized the way we propose novel proteins and the types of challenges we are able to tackle. While these tools have dramatically increased the hit rate of functional designs, zero-shot generations often require further optimization to enhance the desired properties. We present a novel method for generating high fidelity diverse combinatorial libraries. Traditional library generation methods rely on error prone PCR or natural fragment shuffling to generate diversity. Such approaches do not take into account the filtering of designs through structure prediction models or other desired oracles. We demonstrate the ability to post-train inverse folding models to design fragments with filter pass rates of 93.4% and libraries which can easily scale beyond 10^{12} individually addressable variants. Our post-training framework enables control over the search space by centering the library around a reference sequence with tunable mutation distance. Once designed, we demonstrate how individual variants are assembled with simple molecular biology techniques (golden gate assembly). Our cell-free assay platform enables us to screen enzymes at the scale of 10^3 measurements per day, each with replicates. The libraries generated are easily much larger than most traditional and modern high throughput assays are able to screen, so we demonstrate how the library can be sub sampled smartly to optimize enzymes. For a de novo designed PETase we demonstrate our platform finds a variant with 250x increase in catalytic activity.

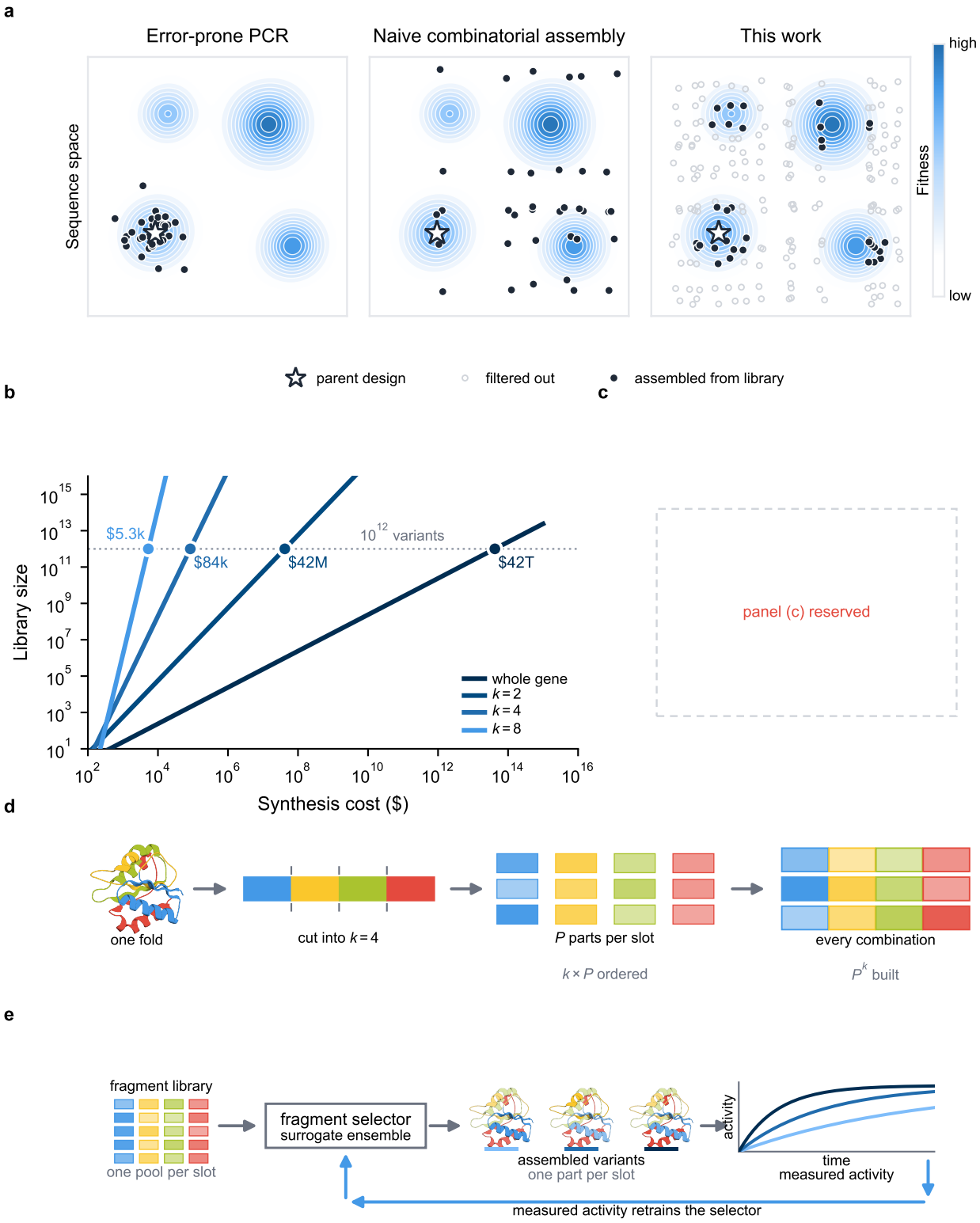

a
Error-prone PCR
Naive combinatorial assembly
This work
Sequence space
Fitness
high
low
parent design
filtered out
assembled from library
b
c
panel (c) reserved
Library size
$5.3k
$84k
$42M
$42T
10^12 variants
whole gene
k = 2
k = 4
k = 8
Synthesis cost ($)
d
one fold
cut into k = 4
P parts per slot
k × P ordered
every combination
P^k built
e
fragment library
one pool per slot
fragment selector
surrogate ensemble
assembled variants
one part per slot
activity
time
measured activity
measured activity retrains the selector

Figure 1: **A combinatorial library, and the loop around it.** [PLACEHOLDER — the prose is yours. Kept short so the float fits; the facts below are the ones a reader cannot get from the picture.] **(a)** One toy landscape searched three ways with the same number of variants built and screened ($N = 36$). Illustrative: no axis is measured. **(b)** eBlocks are billed at a 300-bp minimum, which binds from $k = 2$ on, so past that point cutting finer stops buying shorter DNA and only buys more pieces of it. 10^{12} variants costs \$42T as whole genes, \$42M at $k = 2$, \$84k at $k = 4$, \$5.3k at $k = 8$. **(c)** [Reserved — to be filled in.] **(d)** What makes the library combinatorial. One fold is cut at $k - 1$ boundaries; each slot is then an independent pool of P parts; and any one part per slot assembles a full-length design. Ordering $k \times P$ fragments therefore builds P^k sequences, which is the exponent panel (b) prices. Parts per slot are drawn as three to keep the columns legible. **(e)** The experimental loop. The arrow returns to the *selector*, not the library: parts are synthesised once and each round changes only which of them get combined.

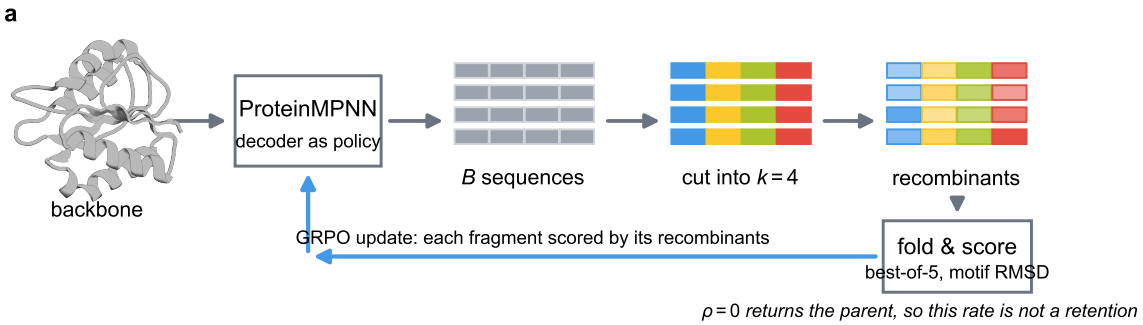


Figure 2: **[Figure 2 — title is yours.]** **(a)** The method. A backbone conditions the ProteinMPNN decoder, which is treated as a policy; the sequences it emits are cut at the fragment boundaries, recombined across parents, and folded best-of-5. Each fragment is scored by the recombinants it appears in, and that per-fragment reward drives the GRPO update. The policy is therefore optimised for how well its fragments *recombine*, not for how good any single sequence is.